

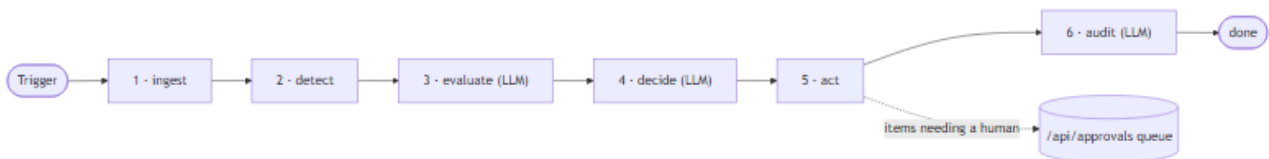
BETSY — THE PIPELINE

How one procurement check runs, from start to finish

WHAT THIS IS

The pipeline is the simpler of Betsy's two ways of working. Think of it as a checklist that Betsy runs top to bottom, the same way every time: look at the data, find the problems, weigh the options, make a call, do the work, and write down what happened.

It runs as six steps in a fixed order. No step is skipped and no step runs out of turn. Even if a step trips over an error, Betsy still reaches the last step, so every run leaves a written record behind.



See the picture: [diagrams/pipeline-overview.mmd](#)

THE SIX STEPS, IN PLAIN WORDS

1. **INGEST** — gather the data. Betsy pulls in the current stock levels, the supplier list, the past purchase orders, and the invoices. All the reading happens here, once, so the later steps just work with what was collected.

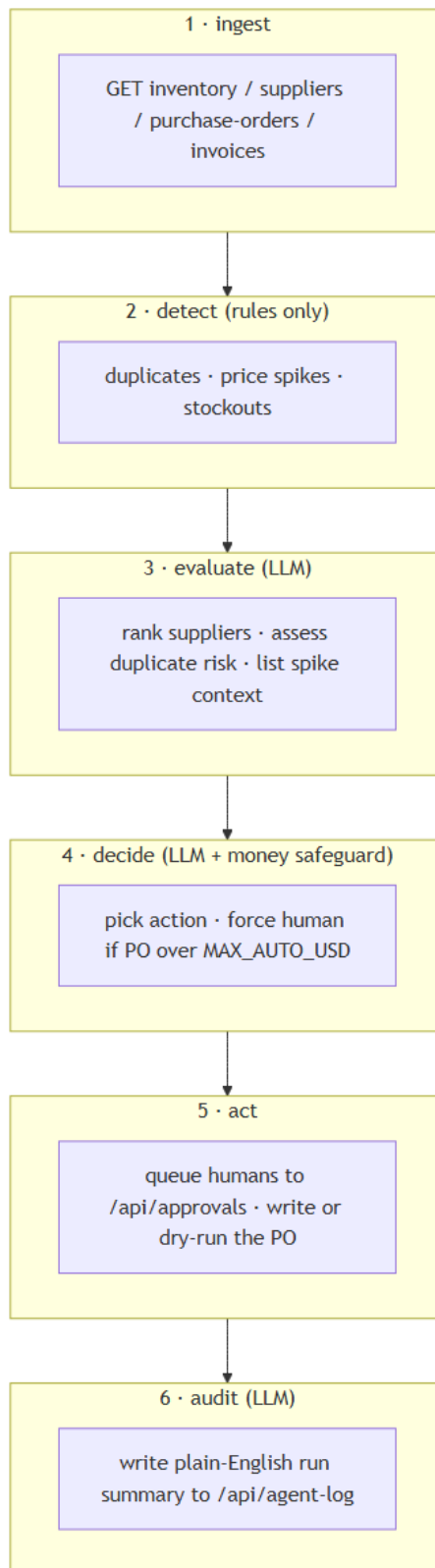
2. **DETECT** — find every problem. Betsy scans the data for three kinds of trouble: stock about to run out, a price that has jumped far above normal, and invoices that look like duplicates. This step uses plain rules (no AI yet), so it is fast and gives the same answer every time. It lists ALL problems it finds before anything is acted on.

3. **EVALUATE** — weigh the options. For each problem, Betsy works out what the choices are. For a low-stock item it ranks the suppliers (a supplier that is both reliable and fast scores best) and asks the AI to recommend one and explain why. For a price jump or a duplicate invoice it gathers the facts a person would need. If the AI is unavailable, a simple rule stands in for it.

4. **DECIDE** — make the call, and respect the money limit. Betsy turns each option into a firm decision. This is also where the safety limit is enforced: any purchase over the set dollar limit is automatically handed to a person, no matter what the AI suggested. Duplicate invoices and price jumps always go to a person too.

5. **ACT** — carry it out. Anything Betsy is allowed to do on its own, it does. Anything that needs a person is parked in the approvals queue with the paperwork already filled in (see "The human checkpoint" below). By default Betsy runs in a safe "practice" mode and does not place real orders until that mode is switched off.

6. **AUDIT** — write it down. Betsy writes a short, plain-English summary of the whole run so anyone can read what happened and why. This step always runs, so even a run that hit a problem still ends with a record.



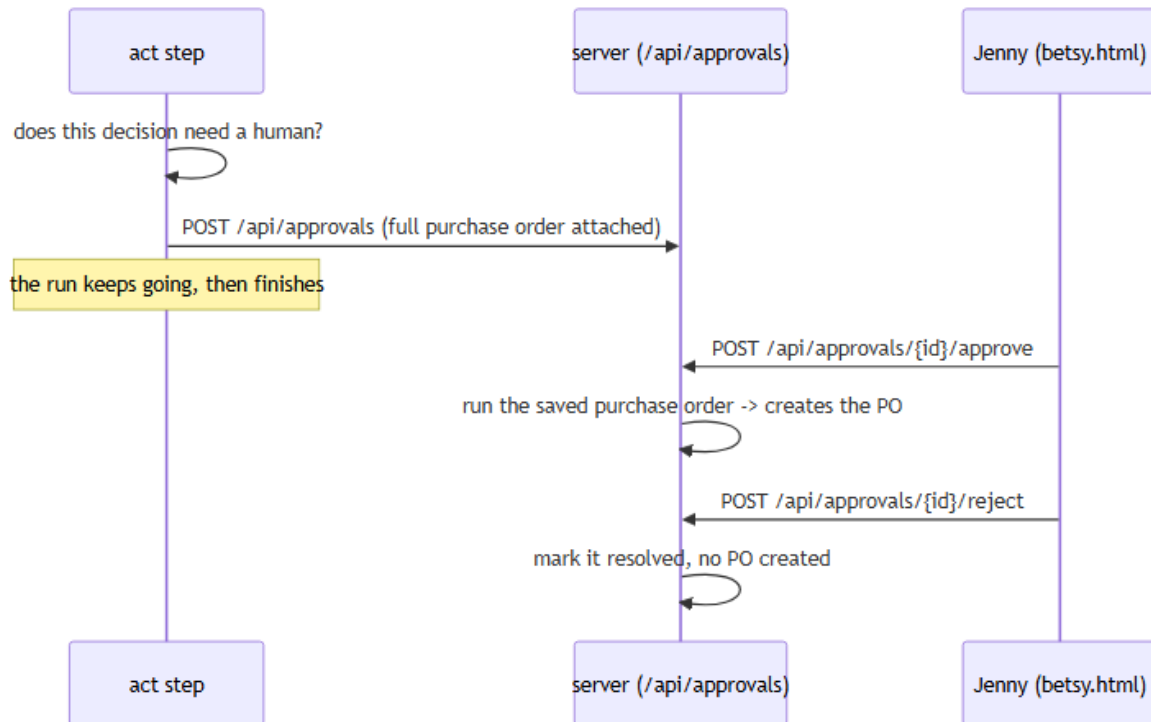
See the picture: *diagrams/pipeline-dataflow.mmd*

THE HUMAN CHECKPOINT (how approval works)

Betsy does not freeze and wait for a person in the middle of a run. Instead, when a decision needs a human, Betsy drops it into an approvals queue — with the full purchase

order already prepared — and finishes the run. Later, Jenny opens her screen and either approves or rejects it.

Because the purchase order is saved at the moment it is queued, approving it simply runs that saved order. Betsy does not re-think or re-price anything on approval, so what Jenny sees is exactly what gets done.



See the picture: [diagrams/pipeline-approval.mmd](#)

WHAT HAPPENS IF A STEP BREAKS

If a step hits an error, it does not crash the whole run. It quietly records the problem and hands an empty result to the next step. The run keeps moving down the checklist and still reaches the final "write it down" step. So a broken run is still a recorded run — you can always see what went wrong.

THE MONEY SAFETY LIMIT

There is one spending limit: a ceiling on how big a single purchase Betsy may approve on its own (set to \$5,000 by default, and changeable without touching the code). Anything above that is sent to a person. There is no separate daily total limit — just this one per-order ceiling.

WHY IT'S BUILT THIS WAY

The before/after story in the GAP analysis shows Jenny doing all of this by hand — checking stock, emailing suppliers, comparing quotes, waiting for sign-off, chasing deliveries, matching invoices — spread across two to three days. The pipeline squeezes that same chain into one quick automated pass. The money limit and the approvals queue keep the single human check that the GAP's "after" picture still wants: a person on high-value and unusual decisions, and nothing else.

See: docs/gap-analysis.txt (and the styled version, bpm_analysis.html)

WHERE THIS LIVES IN THE CODE

- pipeline/graph.py builds the six-step run and its fixed order
- pipeline/state.py the shared worksheet the steps pass along
- pipeline/run.py how a run is started (and how to run one step)
- pipeline/nodes/ingest.py step 1, gather the data
- pipeline/nodes/detect.py step 2, find the problems (plain rules)
- pipeline/nodes/evaluate.py step 3, weigh the options (AI + fallback)
- pipeline/nodes/decide.py step 4, make the call + the money limit
- pipeline/nodes/act.py step 5, do it / queue it / practice mode
- pipeline/nodes/audit.py step 6, write the summary
- server/routers/approvals.py the approve/reject screen actions
- shared/llm.py the AI connection (local Ollama, llama3.1:8b)

DIAGRAMS FOR THIS DOCUMENT

- diagrams/pipeline-overview.mmd the six steps, left to right
- diagrams/pipeline-dataflow.mmd the six steps, with what each does
- diagrams/pipeline-approval.mmd how the human checkpoint works

KNOWN ISSUE TO RECONCILE

The "price spike" test scenario describes its best quote (\$13.60) as "50% above" the ~\$11.40 average, but \$13.60 is only about 19% above \$11.40 — which is under the 30% line the code uses to flag a spike. So that scenario might not reliably trigger as written. Worth fixing later by either lowering the 30% threshold or raising the scenario's quoted prices.